RepoMind: Architectural Guide & Concept Manual

Understanding Knowledge Graphs, GraphRAG, NetworkX, AST, and What-If Analysis in Simple & Technical Terms

Document Purpose:
This guide explains exactly how RepoMind works under the hood. For every major concept (Knowledge Graphs, NetworkX, AST, FAISS, GraphRAG, and What-If Analysis), this document provides both a Simple Real-World Explanation (easy to grasp and explain to anyone) and a Technical Deep-Dive (exact algorithms, data structures, and implementation details).

1. What is RepoMind?

RepoMind Overview

In Simple Words (Intuitive Analogy)	In Technical Words (Under the Hood)
Most AI coding assistants (like Copilot) can write or explain a single snippet of code. But they are blind to how the whole project connects together. If you change one file, they cannot tell you what else will break. RepoMind is an AI architect that understands the entire codebase as an interconnected map and predicts the exact consequences of changing any piece of software.	RepoMind is an Explainable GraphRAG software intelligence system. It ingests public GitHub repositories, performs static AST analysis to construct an explicit directed multigraph (NetworkX), indexes code chunks into a dense vector index (FAISS), and couples both with a dual-hop retrieval engine to perform grounded What-If change-impact analysis and blast-radius tracing.

2. What is a Software Knowledge Graph?

Knowledge Graph

In Simple Words (Intuitive Analogy)	In Technical Words (Under the Hood)
Imagine a social network where people are nodes and friendships are edges. In software, a Knowledge Graph treats functions, classes, and files like people, and imports or function calls like relationships. If Class A calls Class B, there is a one-way bridge between them. This allows us to walk across the bridges to see who depends on who.	A Knowledge Graph G = (V, E) is a directed labeled multigraph where vertices V represent software entities (Modules, Files, Classes, Functions, Methods, Configs, Libraries, Tests) and directed edges E represent typed semantic relationships (CONTAINS, IMPORTS, CALLS, DEPENDS_ON, IMPLEMENTS, CONFIGURES, DOCUMENTS, TESTS). Every node and edge retains metadata (source line

	numbers, file path, confidence, is_inferred).

3. Python AST (Abstract Syntax Tree) — The Scanner

Python AST

In Simple Words (Intuitive Analogy)	In Technical Words (Under the Hood)
Think of an AST as an X-ray scanner for code. We don't run the code or execute it (which could be dangerous or crash). Instead, Python reads the grammar of the code and breaks it down into a tree structure showing every class, function, parameter, and function call clearly.	Python's standard library module `ast` parses source code into an abstract syntax tree adhering to the Python grammar grammar specification. RepoMind implements an `ASTVisitor` (subclassing `ast.NodeVisitor`) that intercepts `ast.ClassDef`, `ast.FunctionDef`, `ast.Import`, `ast.ImportFrom`, and `ast.Call` nodes to deterministically extract entity declarations and caller-callee bindings with exact line coordinates.

4. NetworkX — The Graph Brain

NetworkX

In Simple Words (Intuitive Analogy)	In Technical Words (Under the Hood)
NetworkX is our in-memory GPS navigation system for code. Once AST finds all the functions and calls, NetworkX stores them and lets us ask questions like: 'Starting from FAISS, what are all the paths that lead backwards to find who is calling it?' It can find shortcuts, loops, and calculate which components are the most important hubs in the system.	NetworkX is a high-performance Python graph theory library. RepoMind uses `nx.DiGraph` to represent the software topology. It enables O(V + E) Breadth-First Search (BFS) traversals via `single_source_shortest_path_length`, topological sorting, in-degree/out-degree centrality calculations, cycle detection, and subgraph slicing for targeted What-If blast radius analysis.

5. FAISS & Vector Embeddings — The Semantic Search

FAISS & Embeddings

In Simple Words (Intuitive Analogy)	In Technical Words (Under the Hood)
Computers cannot understand English text or Python code directly, so an embedding engine translates code chunks into lists of numbers (vectors) based on their meaning. FAISS (built by Facebook AI) is a super-fast search engine that compares these lists of numbers to find the most	FAISS (Facebook AI Similarity Search) is an indexed similarity search library. RepoMind chunks the repository using structure-aware AST boundaries, converts text to 128/256-dimensional unit-normalized dense vectors using TF-IDF + TruncatedSVD (or SentenceTransformers), and stores them in a `faiss.IndexFlatIP` index where vector

relevant code chunks in milliseconds.	inner product corresponds directly to cosine similarity.

6. Standard Vector RAG vs. GraphRAG vs. Hybrid GraphRAG

◇ GraphRAG vs Standard RAG

In Simple Words (Intuitive Analogy)	In Technical Words (Under the Hood)
Standard RAG is like using Ctrl+F keyword search: if you ask 'What happens if I change FAISS?', it only finds files that literally have the word 'FAISS'. But it completely misses the API routes or pipelines that call FAISS indirectly! GraphRAG follows the actual call bridges to find everyone affected. Hybrid GraphRAG combines both: it finds the starting files using meaning, and then walks the graph bridges to find all hidden callers.	Standard Vector RAG computes cosine similarity between a query vector and chunk vectors, retrieving top-k chunks. It suffers from the 'horizon problem'—failing to retrieve topologically connected components that lack lexical or semantic overlap with the prompt. Hybrid GraphRAG performs vector retrieval, extracts the identified entities into the NetworkX graph, executes a k-hop bidirectional BFS upstream and downstream, and merges both contexts for grounded synthesis.

7. The What-If Analysis Engine (The Core Differentiator)

◇ What-If Analysis Engine

In Simple Words (Intuitive Analogy)	In Technical Words (Under the Hood)
Imagine you want to replace your car's petrol engine with an electric motor. You need to know: Will the transmission fit? Will the dashboard gauges still work? What wires need to be changed? RepoMind's What-If engine does this for software. When you ask 'What if we replace FAISS with ChromaDB?', it identifies FAISS, checks every pipe connected to it, finds what breaks, identifies which tests must be rewritten, and outputs a complete risk checklist.	The What-If Engine operates in 4 stages: 1. Intent Parsing: `ChangeParser` extracts the action (`REPLACE`), target (`FAISS`), and replacement (`ChromaDB`). 2. Neighborhood Traversal: `ImpactAnalyzer` traverses reverse edges in `nx.DiGraph` to find 1-hop direct callers (High Risk), 2-to-k hop transitive callers (Medium Risk), and linked test functions via `TESTS` edges. 3. Subgraph Isolation: Extracts the impacted subgraph for visual rendering. 4. Grounded Synthesis: Feeds verified paths into the LLM with strict negative constraints to prevent hallucination, generating an explainable impact report.

8. End-to-End Data Pipeline

Here is the exact step-by-step path a repository takes through RepoMind:

• **Step 1: URL Input & Ingestion:** User provides a GitHub URL. GitHub REST API fetches README, file tree, dependencies, and code, saving them to local cache.

• **Step 2: Structure-Aware Chunking:** Python AST parses files into clean semantic chunks (by Class, Function, and Method) with line numbers.

• **Step 3: Knowledge Graph Assembly:** EntityExtractor and RelationshipExtractor build NetworkX nodes (Classes, Funcs, Libs) and edges (CALLS, IMPORTS, TESTS).

• **Step 4: Vector Indexing (FAISS):** Code chunks are converted into normalized dense vectors and stored in a local FAISS index.

• **Step 5: Hybrid Retrieval:** When a query arrives, FAISS finds semantic matches while NetworkX traverses caller bridges to gather full context.

• **Step 6: What-If Blast Radius Tracing:** For hypothetical refactorings, reverse graph BFS identifies direct callers, transitive callers, and tests.

• **Step 7: Interactive UI & Reports:** Streamlit displays interactive PyVis physics networks, verified citations, risk badges, and downloadable reports.